Übungsblatt 7: Tracing und Timing-Analyse

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Ziele und Ablauf

Sie definieren Messintervalle, rechnen Zeitgrößen und beurteilen Stichproben und Aufzeichnungslücken. Grundlage sind Vorlesungen 1–7. Ein Trace ist eine geordnete Aufzeichnung ausgewählter Ereignisse. Ein Messvertrag benennt Anfang, Ende, Quelle, Einheit und Bedingungen.

Verbindlicher Übungskern ist die Klausurvorbereitung auf Papier. Die angegebenen Punkte dienen zur Selbsteinschätzung; die Modulnote entsteht durch die Papierklausur. Alle Rechen- und Analyseaufgaben sind mit den angegebenen Daten ohne Hardware oder erfolgreiche frühere Programmieraufgaben lösbar. Begründen Sie Entscheidungen und nennen Sie Modellannahmen.

Abschnitt	Minuten	Zweck
Papierkern mit Besprechung	50	32 Orientierungspunkte
Transfer und Besprechung	15	Rechnen und begründen
Praxisfall: vorbereitete Demonstration oder Papieralternative	15	Vorhersagen, erklären, Grenzen nennen

Der Übungsblock umfasst **80 Minuten**. Die Praxisphase umfasst etwa fünf Minuten Vorhersage, höchstens fünf Minuten Demonstration und fünf Minuten Erklärung. Ohne funktionierenden Aufbau bearbeiten Sie in denselben 15 Minuten den vollständig vorgegebenen Papierfall. **Nach spätestens drei Minuten technischer Fehlersuche wird zum Papierfall gewechselt.** Dadurch bleibt der Übungsblock auch bei Setup-Problemen nutzbar.

Für eine Demonstration verwendet die Lehrperson **STM32CubeMX**, **VS Code mit STM32 VS Code Extension** und **NUCLEO-H755ZI-Q / STM32H755ZIT6**. Sie bereitet die geprüften Projektvarianten vor dem Termin vor. Installation, Projektgenerierung, Integration, Regenerierungsprüfung, Build und Flash sind keine Pflichtaufgaben im Übungstermin. Eine eigene Implementierung ist freiwillige Vertiefung. Alle Zahlen dieses Blatts sind Lehrannahmen, soweit sie nicht ausdrücklich als eigene Messwerte gekennzeichnet sind.

Der Übungsblock ergänzt 80 Minuten Erklärung und 20 Minuten Quiz/Abschluss zum Termin mit 180 Minuten.

Teil A: Papieraufgaben

Aufgabe 1: Ausgabe und Beobachtereffekt · 8 Punkte

Eine Funktion erledigt in einem vereinfachten Modell 10 µs eigene Arbeit. Danach gibt sie die fünf ASCII-Zeichen `Hallo` bei 115200 bit/s über UART aus: ein Startbit, acht Datenbits, keine Parität, ein Stopbit je Byte. Kein Nullterminator oder Zeilenumbruch wird übertragen. Relative Abschlussfrist ab Funktionsbeginn: 50 µs.

1. Berechnen Sie Leitungsbitzahl und reine Leitungsdauer. (2 P)
2. Im **blockierenden Lehrfall** liegt die volle Leitungsdauer zusätzlich innerhalb des Antwortintervalls, weitere Kosten sind null. Berechnen Sie Antwortzeit und Fristabweichung. (2 P)
3. Warum wird dieselbe Leitungsdauer bei IRQ- oder DMA-Ausgabe nicht automatisch zur gleichen CPU-Zeit? Welche Kosten oder Wartephasen können trotzdem verbleiben? (2 P)
4. Definieren Sie einen GPIO-Messvertrag um die ursprüngliche Funktion. Nennen Sie zwei Gründe, warum ein längerer Puls keine reine Funktions-CPU-Zeit belegt, und eine geeignete Position für die Ausgabe. (2 P)

Musterlösung Aufgabe 1

Je Byte zehn Bits, also $5 \cdot 10 = 50$ Bit. Die Leitungsdauer ist $50/115200 = 0{,}0004340278$ Sekunden, etwa 434,03 µs. UART bedeutet Universal Asynchronous Receiver/Transmitter, serielle Schnittstelle. Diese Zahl ist keine Formatierungsdauer und kein Chiphalt.

Im vorgegebenen blockierenden Modell: $R = 10 + 434{,}03 = 444{,}03$ µs. Die Frist 50 µs wird um etwa 394,03 µs überschritten. Anders definierte Ausgabewege dürfen nicht einfach dieselbe CPU-Rechnung übernehmen.

IRQ bedeutet Interrupt Request, Interruptanforderung, DMA Direct Memory Access, Hardware-Speicherzugriff. Senderhardware kann die Leitung bedienen, während die CPU andere Arbeit erledigt. Formatierung, Pufferverwaltung, Sperren, Start-/Abschlussbehandlung und gegebenenfalls Warten auf freien Platz bleiben. DMA benötigt außerdem eine gültige Pufferlebensdauer.

GPIO bedeutet General-Purpose Input/Output. Der Messvertrag läuft von der physischen steigenden bis zur physischen fallenden Flanke am geprüften Pin. Unterbrechungen zwischen den Marken, Weg über Bus/Pad und Auflösung des Messgeräts beeinflussen das Intervall. Es ist keine direkte Store-zu-Store-Zeit. Die Ausgabe liegt nach der Abschlussmarke, außerhalb des Fensters; ihre mögliche Wirkung auf spätere Jobs wird trotzdem dokumentiert.

Aufgabe 2: Jobspur und Interruptkette · 8 Punkte

Jobmodell: A wird bei 1 ms freigegeben, läuft 2–4 ms und 6–7 ms und ist bei 7 ms fertig. Von 4–6 ms erledigt der Prozessor fremde Arbeit. Absolute Deadline: 8 ms. Kontextwechselkosten sind in diesem Modell null.

1. Berechnen Sie eigene CPU-Zeit, Antwortzeit, Startwartezeit und Slack. Wird die Frist gehalten? Warum ist „fertig“ nicht automatisch das Ende der Tasklebensdauer? (4 P)

Interruptmodell, gemeinsame hypothetische Uhr:

Ereignis	Zeit in µs
externe Flanke	100

Ereignis	Zeit in µs
ISR-Eintritt	106
Veröffentlichung	110
Task Ready	111
ISR-Ende	115
Taskstart	120
geforderte Reaktion	128

2. Berechnen Sie Eintrittslatenz, ISR-Intervall, Ready-Wartezeit und Gesamtreaktion. Geben Sie eine disjunkte Zerlegung der Gesamtreaktion an und begründen Sie, warum die vier Kennzahlen nicht miteinander aufsummiert werden. (4 P)

Musterlösung Aufgabe 2

Eigene Arbeit: $(4-2)+(7-6)=3$ ms. Antwortzeit: $R=f-r=7-1=6$ ms. Erstes Ready-Warten: $s-r=2-1=1$ ms. Slack: $d-f=8-7=1$ ms, die Frist wird gehalten. Prüfung: Anfangswarten eins plus Verdrängung zwei plus eigene Arbeit drei sind sechs. Fremde Arbeit zählt nicht als eigene A-Arbeit. Ein Job ist eine Aktivierung; seine Task kann danach für weitere Aktivierungen bestehen bleiben. Die drei Millisekunden sind kein WCET-Nachweis.

ISR bedeutet Interrupt Service Routine, Behandler. Eintritt: $106-100=6$ µs. ISR-Intervall: $115-106=9$ µs. Ready bis Start: $120-111=9$ µs. Gesamtreaktion: $128-100=28$ µs. Eine disjunkte Zerlegung lautet $(106-100)+(110-106)+(111-110)+(120-111)+(128-120)=6+4+1+9+8=28$ µs. Ready-Warten 111–120 überlappt das ISR-Intervall bis 115. Die volle ISR-Dauer darf nicht nochmals hinzugefügt werden. Der Taskstart bei 120 ist außerdem nicht schon die Reaktion bei 128.

Aufgabe 3: Lokaler Zähler und Aussagegrenze · 8 Punkte

DWT-CYCCNT ist hier ein vorzeichenloser 32-Bit-Zähler eines Kerns. Seine Unterstützung und sein Lauf seien für das Modell bestätigt.

1. Anfang `0xFFFFFFF0`, Ende `0x00000030`: Berechnen Sie die vorzeichenlose Modulo-Differenz. Bei hypothetisch stabilen 400 MHz: Welche Zeit entspricht ihr und wie lang ist ein voller Umlauf? Warum genügt die Differenz allein nicht für ein beliebig langes Intervall? (3 P)
2. Tausend Takte werden einmal bei 400 MHz und einmal bei 200 MHz gezählt. Rechnen Sie beide Zeiten. Darf man den Endstand des M4 vom Anfangsstand des M7 subtrahieren? Welche Voraussetzungen fehlen? (2 P)
3. Eine Stichprobe vollständig definierter Intervalle enthält 12, 15, 14 und 18 µs. Berechnen Sie Anzahl, Minimum, beobachtetes Maximum und Mittelwert. Warum ist das Maximum keine WCET? Warum kann ein CYCCNT-Fenster um eine Funktion auch fremde Arbeit enthalten? (3 P)

Musterlösung Aufgabe 3

DWT bedeutet Data Watchpoint and Trace, CYCCNT Cycle Count. Sechzehn Takte bis zum Umlauf plus achtundvierzig danach ergeben 64 modulo 2^{32}. $64/(400\cdot10^6)=0{,}16$ µs. Ein voller Umlauf umfasst 4294967296 Takte, also etwa 10,73741824 s bei 400 MHz. Die Differenz ist identisch, wenn zusätzliche ganze Umläufe vergangen sind. Die Umrechnung benötigt einen belegten Zeitraum kürzer als einen Umlauf, stabilen Takt und fortlaufenden Zähler; Halt oder Sleep können diese Annahme verletzen.

1000 Takte entsprechen 2,5 µs bei 400 MHz und 5 µs bei 200 MHz. Die Lehrfrequenzen sind keine bestätigten Boardwerte. Zwei Kernzähler haben möglicherweise verschiedene Nullpunkte, Frequenzen und Halt-/Sleepbedingungen. Ihre Differenz benötigt eine bestätigte gemeinsame Skalierung und Zeitkorrelation; eine Dual-Core-Debugverbindung reicht nicht.

Anzahl vier, Minimum zwölf, Maximum achtzehn, Mittelwert $59/4 = 14{,}75$ µs. WCET ist Worst-Case Execution Time, eine begründete obere Schranke für alle zulässigen Ausführungen unter Annahmen. Ungetestete Pfade oder Zustände können länger sein. Ein bei Unterbrechung weiterlaufender Zähler zählt die dazwischenliegende ISR- oder andere Taskarbeit mit. Eigene CPU-Zeit benötigt eine passende Running-/Exceptionzuordnung.

Aufgabe 4: Quelle, Kapazität und Datenqualität · 8 Punkte

1. Ein konfiguriertes DWT-Ereignis zeigt einen M7-Store mit eins. Der Speicherbereich ist tatsächlich Write-Back-cachebar, M7 hat eine Dirty-Kopie, SRAM kann noch null enthalten. Beweist das Ereignis den vom M4 gelesenen Wert? Was muss bei einem Filter auf `ipc.payload` zusätzlich geprüft werden? Das Layout aus Vorlesung 6 hat Offsets 0/4/8. (2 P)
2. Während eines M4-Interrupts zeigt `pxCurrentTCB` weiter auf Task A. Läuft damit A-Code? Unterscheiden Sie Taskliste, Running-Profil und Zustandshistorie. (2 P)
3. Synthetisches Ratenmodell: 250000 Ereignisse/s mit vier Byte je Ereignis, Abfluss 600000 Byte/s, anfangs freier Puffer 64 KiB. Kein Paketoverhead, konstante Raten. Wann ist der Puffer voll? Würde doppelte Kapazität die dauerhafte Überlast lösen? (2 P)
4. Im Trace fehlt ein Anfangspaket. Später enthält ein Abschnitt eine Verlustmarke und danach bestätigte neue Synchronisation mit passendem Kontext. Darf der Anfang als null gezeichnet und die Lücke interpoliert werden? Wie behandeln Sie den späteren vollständigen Abschnitt? (2 P)

Musterlösung Aufgabe 4

Ein M7-Storeereignis belegt einen Zugriff der ausgewählten Quelle, nicht automatisch den aktuellen SRAM-Wert oder die M4-Sicht. Dirty bedeutet lokal verändert, Write Back spätere Rückschreibung. Geprüft werden Kern, tatsächliches Image/Map, Basisadresse, Feldbreite, Offset acht für payload, Richtung und verfügbare Komparatorfunktion. Offset vier wäre state und eine andere Frage. Ein Timeout überträgt keinen Besitz am Block.

pxCurrentTCB ist der Zeiger auf den Task Control Block, Verwaltungsblock der ausgewählten Task. Während der ISR kann der Kern Exceptioncode ausführen. Eine Taskliste ist eine Momentaufnahme. Ein korrektes Running-Profil benötigt Task-/Exceptionzuordnung. Eine Ready-/Blocked-Historie benötigt zusätzlich erfasste Zustandsereignisse. ISR und PendSV werden nicht als eigene A-Arbeit gezählt.

Eingang $250000 \cdot 4 = 1000000$ Byte/s. Wachstumsrate $1000000 - 600000 = 400000$ Byte/s. $64 \cdot 1024 = 65536$ Byte. Füllzeit $65536/400000 = 0{,}16384$ s. Doppelte Kapazität ergibt hier 0,32768 s, beseitigt aber die positive dauerhafte Wachstumsrate nicht. Das Verhalten bei Vollbelegung hängt vom tatsächlichen Pfad ab.

Fehlender Anfang heißt unbekannt, nicht null. Ein Verlustabschnitt wird markiert und nicht mit einer erfundenen Geraden geschlossen. Erst nach neuer bestätigter Synchronisation, Kontext- und Zeitzuordnung kann ein späterer vollständiger Abschnitt separat gelten. Summen oder vollständige Jobantworten werden nicht ungeprüft über die Lücke gebildet.

Transfer: Was widerlegt eine Garantie? · 15 Minuten

Bearbeiten Sie zehn Minuten und vergleichen Sie fünf Minuten. Keine zusätzlichen Punkte. Vier vollständige Jobantwortintervalle aus einer gemeinsamen **Modelluhr** betragen 38, 42, 55 und 41 µs. Die relative Deadline ist 50 µs. Anfänge und Abschlüsse sind vollständig erfasst; es gibt in diesem Abschnitt keine Lücke.

1. Berechnen Sie Mittelwert und Maximum. Ist die Frist in allen vier Fällen erfüllt?
2. Ein Review meint: „Die Stichprobe beweist keine WCET, deshalb zählt der 55-µs-Fall nicht als Widerlegung.“ Beurteilen Sie das Argument. Unterscheiden Sie dabei Antwortzeit von eigener CPU-Zeit.
3. In einem anderen Trace fehlt der Abschluss des Jobs. Wie wird dieser Fall beurteilt, ohne ihm null oder einen erfundenen Höchstwert zuzuweisen?

Musterlösung Transfer 7

Mittelwert $176/4 = 44\,\mu s$, beobachtetes Maximum 55 µs. Drei Fälle erfüllen die Frist, einer verletzt sie um 5 µs. Eine gültige Fristverletzung ist ein Gegenbeispiel gegen die Garantie für diese Bedingungen. Dazu muss die Stichprobe nicht bereits alle Worst Cases abdecken.

Die Intervalle sind Jobantwortzeiten und können Wartezeiten sowie fremde Arbeit enthalten; sie sind keine WCET der eigenen CPU-Arbeit. Eine Reihe günstiger Intervalle beweist keine allgemeine Schranke, ein gültiger ungünstiger Fall kann eine behauptete Schranke aber widerlegen.

Beim fehlenden Abschluss ist die vollständige Antwortzeit nicht bestimmbar. Der Prüfstatus lautet nicht auswertbar beziehungsweise nicht geprüft; der Fall wird als fehlendes Paar ausgewiesen und weder als null noch als erfolgreich behandelt.

Teil B: Praxisfall · 15 Minuten

Praxisfall 7: Ein Zählerfenster auswerten

Papieralternative, synthetische Zählerdaten: Ein bestätigter lokaler Zähler läuft bei konstant 200 MHz, ohne Halt oder Umlaufmehrdeutigkeit. Jeder Eintrag umfasst die Zeit zwischen zwei Zählerlesungen, einschließlich Barrieren und möglicher Interrupts. Logging liegt außerhalb. Drei EMPTY-Fenster ergeben 200, 220 und 240 Takte; drei WORK-Fenster 2000, 2400 und 2200 Takte.

1. Berechnen Sie Minimum, Maximum und Mittelwert je Gruppe in µs sowie die WORK-Spanne.
2. Ein Review zieht von jedem Arbeitswert das kleinste Leerfenster ab und nennt den größten Rest „WCET der Funktion“. Welche zwei Nachweislücken bleiben?
3. Welche Angaben brauchen Sie zusätzlich, wenn ähnliche Zahlen aus einem echten Boardlauf stammen?

Bei vorbereitetem Board: Die Lehrperson zeigt einen abgeschlossenen `lab07_timing`-Lauf und die beiden Gruppen. Der reale Baustein hat sieben Einträge je Gruppe; die drei Papierwerte sind ein unabhängiges Lehrmodell. Neue Tracehardware und ein BlueBox-/winIDEA-Aufbau sind freiwillige Vertiefung.

Musterlösung Praxisfall 7

EMPTY: 1, 1,1, 1,2 µs; Minimum 1, Maximum 1,2, Mittelwert 1,1 µs. WORK: 10, 12, 11 µs; Minimum 10, Maximum 12, Mittelwert 11 µs und Spanne 2 µs.

Ein fremder minimaler Leerwert beschreibt nicht automatisch die Kosten jedes Arbeitsfensters. Zudem können die Arbeitsfenster fremde Interruptarbeit enthalten und bilden nur eine endliche Stichprobe. Auch der größte Restwert 11 µs ist damit keine WCET der eigenen Funktion. Ein echter Lauf benötigt insbesondere geprüften Takt, fortlaufenden Zähler, Umlaufgrenze, Quelle, Cache-/Last-/Interruptbedingungen und den Debugzustand.

Ergebnis und freiwillige Vertiefung

Halten Sie die begründeten Papierlösungen (32 Orientierungspunkte), Ihre Transferantworten und die Erklärung des Praxisfalls fest. Projektdateien, Versionslisten, Flashnachweise und Messprotokolle sind keine Pflichtabgabe dieses Übungsblocks. Für das Verständnis des nächsten Blatts ist keine erfolgreiche Firmwareintegration erforderlich.

Die freiwillige Programmiervertiefung enthält die bisherige technische Vorbereitung und Integration. Sie richtet sich an die Lehrperson sowie an Studierende, die außerhalb des Termins mehr Praxis wünschen. Ein fehlender Boardlauf beeinträchtigt die Bearbeitbarkeit der Papieraufgaben nicht. In eigenen Protokollen bleiben ungemessene Zeitfelder leer; beobachtete Stichprobenmaxima sind keine allgemeine Worst-Case-Garantie.

Quellen: Vorlesung 7; Giorgio C. Buttazzo, *Hard Real-Time Computing Systems*, 3. Auflage; ST RM0399, UM2408 und AN4989; tatsächliche CMSIS-M7-Header, insbesondere DWT-Register und ITM_SendChar; Dokumentation der installierten STM32 VS Code Extension und gegebenenfalls des vorhandenen winIDEA-/BlueBox-Aufbaus.